Desarrollo de Aplicaciones con Objetos

Córdoba, 2026.

Material conceptual

Interfaces graficas en python.

Introducción

Para crear formularios en pantalla en Python, puedes utilizar bibliotecas gráficas como Tkinter, PyQt, wxPython y Kivy. Cada una de estas bibliotecas tiene sus propias ventajas y desventajas, por lo que puedes elegir la que mejor se adapte a tus necesidades y preferencias.

A continuación, una breve introducción a cómo utilizar la biblioteca Tkinter para crear formularios simples en pantalla:

Tkinter:

Tkinter es una biblioteca gráfica incorporada en Python que te permite crear interfaces de usuario de manera sencilla. Aquí hay un ejemplo básico de cómo crear un formulario de entrada de datos con Tkinter:

```
import tkinter as tk

# Función para manejar el botón de enviar
def enviar_datos():
    nombre = entry_nombre.get()
    edad = entry_edad.get()
    print(f"Nombre: {nombre}, Edad: {edad}")

# Crear una ventana principal
ventana = tk.Tk()
ventana.title("Formulario de Datos")

# Crear etiquetas y campos de entrada
label_nombre = tk.Label(ventana, text="Nombre:")
label_nombre.pack()
entry_nombre = tk.Entry(ventana)
entry_nombre.pack()
```

```
# Crear etiquetas y campos de entrada
label_nombre = tk.Label(ventana, text="Nombre:")
label_nombre.pack()
entry_nombre = tk.Entry(ventana)
entry_nombre.pack()

label_edad = tk.Label(ventana, text="Edad:")
label_edad.pack()
entry_edad = tk.Entry(ventana)
entry_edad.pack()

# Crear un botón para enviar los datos
boton_enviar = tk.Button(ventana, text="Enviar", command=enviar_datos)
boton_enviar.pack()

# Iniciar el bucle principal
ventana.mainloop()
```

Este es un formulario muy simple que recopila el nombre y la edad de una persona y los muestra en la consola cuando se hace clic en el botón "Enviar". Tkinter ofrece muchas más capacidades para crear interfaces de usuario más complejas con botones, cuadros de lista, casillas de verificación, etc. Puedes personalizar y diseñar tu formulario según tus necesidades.

Ten en cuenta que Tkinter está disponible por defecto en la mayoría de las instalaciones de Python, por lo que es una excelente opción para comenzar a crear interfaces de usuario simples. Si necesitas funcionalidades más avanzadas o un diseño más atractivo, podrías explorar otras bibliotecas como PyQt o wxPython.

Controles mas utilizados.

Tkinter ofrece una variedad de componentes para crear interfaces gráficas de usuario en Python. Algunos de los componentes más utilizados son:

1. **Ventana (Window)**: La ventana principal de la aplicación donde se colocan todos los demás componentes.

2. **Etiqueta (Label)**: Se utiliza para mostrar texto o una etiqueta en la ventana.
3. **Cuadro de Texto (Entry)**: Permite al usuario ingresar texto, como nombres o contraseñas.
4. **Botón (Button)**: Se utiliza para activar una acción cuando se hace clic en él, como enviar un formulario.
5. **Cuadro de Lista (Listbox)**: Proporciona una lista de elementos donde el usuario puede seleccionar uno o más.
6. **Cuadro de Texto Multilínea (Text)**: Permite la entrada y visualización de texto en varias líneas.
7. **Casilla de Verificación (Checkbutton)**: Permite al usuario seleccionar o deseleccionar una opción.
8. **Botón de Radio (Radiobutton)**: Permite al usuario seleccionar una opción entre varias opciones mutuamente excluyentes.
9. **Barra de Desplazamiento (Scrollbar)**: Se utiliza junto con otros widgets, como cuadros de lista y cuadros de texto, para desplazarse por contenido más largo.
10. **Cuadro de Diálogo (Dialog):** Ventanas emergentes que permiten al usuario tomar decisiones o proporcionar información adicional.
11. **Cuadro de Mensaje (Message):** Muestra un mensaje de texto en una ventana emergente.
12. **Menú (Menu):** Permite crear barras de menú en la parte superior de la ventana.
13. **Canvas:** Un área rectangular donde se pueden dibujar formas gráficas y gráficos.
14. **Frames (Marcos):** Se utilizan para organizar y agrupar otros widgets.
15. **Imagen (PhotoImage):** Permite mostrar imágenes en la ventana.
16. **Reloj (Clock):** Muestra la hora y fecha actual.

Estos son solo algunos de los componentes más comunes en Tkinter. Puedes combinarlos y personalizarlos para crear interfaces de usuario completas y funcionales. La elección de los componentes depende de los requisitos específicos de tu aplicación.

Ventana:

En la biblioteca Tkinter, la ventana principal se crea utilizando la clase **Tk()**, y puedes configurar varias propiedades de esta ventana. Algunas de las propiedades más comunes que puedes establecer en una ventana Tkinter incluyen:

1. **Título de la Ventana (title)**: El título que aparece en la barra de título de la ventana.

```
ventana.title("Mi Ventana")
```

2. **Dimensiones de la Ventana (geometry)**: El tamaño y la posición inicial de la ventana en píxeles. Por ejemplo, "400x300" crea una ventana de 400 píxeles de ancho por 300 píxeles de alto.

```
ventana.geometry("400x300")
```

3. **Ícono de la Ventana (iconbitmap)**: Establece un icono personalizado para la ventana. Debe ser un archivo .ico.

```
ventana.iconbitmap("icono.ico")
```

4. **Color de Fondo (configure)**: Cambia el color de fondo de la ventana.

```
ventana.configure(bg="lightblue")
```

5. **Modo Redimensionable (resizable)**: Permite o impide que el usuario redimensione la ventana en anchura y altura. Los valores posibles son **"1"** (redimensionable) o **"0"** (no redimensionable).

```
ventana.resizable(1, 1) # La ventana es redimensionable en ambas direcciones
```

6. **Posición Inicial (geometry)**: Establece la posición inicial de la ventana en la pantalla. Puedes usar **"+x+y"** para establecer la posición en las coordenadas (x, y).

```
ventana.geometry("+100+50") # Abre la ventana en la posición (100, 50)
```

7. **Eliminar Ventana (destroy)**: Cierra la ventana cuando se presiona la "X" en la esquina superior derecha.

```
ventana.destroy()
```

8. **Función al Cerrar (protocol)**: Permite especificar una función que se ejecutará cuando se cierre la ventana.

```
ventana.protocol("WM_DELETE_WINDOW", funcion_cerrar)
```

9. **Cambiar Cursor (cursor)**: Cambia la apariencia del cursor cuando pasa sobre la ventana.

```
ventana.config(cursor="arrow")
```

Estas son algunas de las propiedades comunes que puedes configurar en una ventana Tkinter. Puedes ajustar estas propiedades según tus necesidades específicas para personalizar la apariencia y el comportamiento de tu ventana.

Cuadro de texto:

Para un cuadro de texto en Tkinter, generalmente se utiliza el widget **Entry**. Aquí están algunas de las propiedades y configuraciones más comunes que puedes aplicar a un cuadro de texto (**Entry**):

1. **Texto Inicial (insert) y Posición del Cursor (insert)**: Puedes establecer un texto inicial dentro del cuadro de texto y controlar la posición inicial del cursor.

```
entry = Entry(ventana)
```

```
entry.insert(0, "Texto Inicial")

entry.icursor(0) # Coloca el cursor al principio
```

2. **Ancho (width) y Altura (height)**: Configura el ancho y la altura del cuadro de texto en caracteres y líneas, respectivamente.

```
entry = Entry(ventana, width=30, height=5)
```

3. **Estado (state)**: Puede establecer el estado del cuadro de texto como "normal" o "disabled". Cuando está deshabilitado, el usuario no puede editar el contenido.

```
entry = Entry(ventana, state="normal") # También puedes usar
"disabled"
```

4. **Fondo (bg) y Color del Texto (fg)**: Puedes cambiar el color de fondo y el color del texto del cuadro de texto.

```
entry = Entry(ventana, bg="lightgray", fg="black")
```

5. **Borde (borderwidth) y Relieve (relief)**: Controla el ancho del borde y el estilo del relieve alrededor del cuadro de texto.

```
entry = Entry(ventana, borderwidth=2, relief="groove")
```

6. **Ocultar el Texto (show)**: Si deseas que el texto ingresado en el cuadro de texto se muestre como un carácter diferente (por ejemplo, contraseñas), puedes usar la opción **show**.

```
entry = Entry(ventana, show="*") # Muestra asteriscos en lugar de
texto
```

7. **Evento de Tecla (bind)**: Puedes asignar una función para que se ejecute cuando se presione una tecla en el cuadro de texto.

```
def tecla_presionada(event): print("Tecla presionada:", event.keysym)
entry.bind("<Key>", tecla_presionada)
```

8. **Obtener Valor (get) y Establecer Valor (insert)**: Para obtener el valor actual en el cuadro de texto y para establecer un nuevo valor, puedes usar los métodos **get()** e **insert()**.

```
valor = entry.get() # Obtiene el valor actual del cuadro de texto
entry.insert(0, "Nuevo Texto") # Establece un nuevo valor
```

Estas son algunas de las configuraciones más comunes que puedes aplicar a un cuadro de texto en Tkinter. Puedes personalizar aún más según tus necesidades específicas.

Para editar el contenido.

El método **insert** del widget **Entry** en Tkinter se utiliza para insertar texto en el cuadro de texto. Tiene dos parámetros:

1. **Posición de Inserción (index)**: Este es el primer parámetro y especifica dónde se insertará el nuevo texto en el cuadro de texto. Puedes especificar la posición de inserción como un número entero o como una cadena de la forma **"line.column"**, donde **line** es el número de línea y **column** es el número de columna. El primer carácter tiene la posición **1.0** en Tkinter.
    - Para insertar texto al principio del cuadro de texto: **entry.insert(1.0, "Texto a Insertar")**
    - Para insertar texto en la posición actual del cursor: **entry.insert(INSERT, "Texto a Insertar")**
    - Para insertar texto al final del cuadro de texto: **entry.insert(END, "Texto a Insertar")**
2. **Texto a Insertar (string)**: Este es el segundo parámetro y representa el texto que deseas insertar en el cuadro de texto.

Borrar el contenido.

Puedes borrar el contenido de un widget **Entry** en Tkinter utilizando el método **delete** para eliminar el texto que está actualmente en el cuadro de entrada. El método **delete** toma dos argumentos:

1. **Índice de Inicio**: Este es el índice del primer carácter que deseas borrar. Puedes especificarlo como una cadena en el formato **"line.column"** o como dos números enteros que representan la línea y la columna.
2. **Índice de Fin**: Este es el índice del carácter después del último que deseas borrar. De nuevo, puede especificarse como una cadena en el formato **"line.column"** o como dos números enteros.

Para borrar todo el contenido del cuadro de entrada, puedes usar **"1.0"** como el índice de inicio (que representa el primer carácter) y **END** como el índice de fin (que representa el final del cuadro de entrada).

Validar texto

Para realizar validaciones de datos en una ventana de texto (Entry) en una aplicación de tkinter, puedes usar el método **validate** junto con la opción **validatecommand**. Aquí hay una breve descripción de cómo puedes hacerlo:

1. **Define una función de validación:** Antes de crear el campo de entrada (Entry), define una función que realizará la validación. Esta función debe tener la firma **(new_value, user_input)**, donde **new_value** es el nuevo valor que se va a establecer en el Entry después de la edición y **user_input** es la entrada actual en el campo.

```python
def validate_input(new_value, user_input):
    # Realiza tus validaciones aquí
    if new_value.isnumeric() and int(new_value) >= 0:
        return True
    else:
        return False
```

def validate_input(new_value, user_input):
    # Realiza tus validaciones aquí
    if new_value.isnumeric() and int(new_value) >= 0:
        return True
    else:
        return False

2. **Crea el campo de entrada:** Crea el campo de entrada (Entry) y establece las opciones **validate** y **validatecommand** para que tkinter realice la validación automáticamente:

```python
from tkinter import *

root = Tk()

vcmd = root.register(validate_input)  # Registra la función de validación

entry = Entry(root, validate="key", validatecommand=(vcmd, "%P", "%S"))
entry.pack()

root.mainloop()
```

from tkinter import *

root = Tk() vcmd = root.register(validate_input) # Registra la función de validación

entry = Entry(root, validate="key", validatecommand=(vcmd, "%P", "%S")) entry.pack()

root.mainloop()

En el ejemplo anterior:

- **validate="key"** le indica a tkinter que debe realizar la validación en cada evento de teclado.
- **validatecommand=(vcmd, "%P", "%S")** especifica la función de validación (**vcmd**) y los parámetros **%P** y **%S**. **%P** representa el valor después de la edición, y **%S** representa el carácter que se va a insertar.

Dentro de la función **validate_input**, puedes realizar tus propias validaciones personalizadas. Si la validación falla, devuelve **False**, lo que evitará que se realice la edición en el Entry.

Puedes adaptar esta lógica de validación según tus necesidades específicas, como validar números, texto, fechas, etc.

Método Mainloop()

En el contexto de aplicaciones gráficas desarrolladas con bibliotecas como Tkinter (una biblioteca de GUI para Python), **mainloop()** es un método importante que inicia el bucle principal del programa. Este bucle principal se encarga de escuchar y responder a eventos, como clics de ratón, pulsaciones de teclas y otras interacciones del usuario, así como de actualizar la interfaz gráfica según sea necesario.

Cuando se llama a **mainloop()**, la aplicación entra en un estado en el que está constantemente esperando eventos y respondiendo a ellos. Esto permite que la interfaz gráfica permanezca interactiva y receptiva para el usuario. Si no se llama a **mainloop()**, la ventana de la interfaz gráfica se abrirá, pero la aplicación no responderá a las interacciones del usuario ni se actualizará.

En resumen, **mainloop()** es esencial para que una aplicación gráfica en Python, creada con bibliotecas como Tkinter, funcione de manera adecuada, ya que inicia el ciclo de eventos que maneja todas las interacciones del usuario.

Método Pack.

En la biblioteca Tkinter de Python, el método **pack()** se utiliza para organizar y colocar widgets (como botones, etiquetas, cuadros de texto, etc.) dentro de un contenedor, generalmente un widget contenedor, como un Frame o una ventana principal. El método **pack()** es una forma de administrar la disposición de los widgets dentro de un contenedor de manera simple y eficiente.

Aquí hay algunos puntos clave sobre cómo funciona el método **pack()**:

1. **Empaquetamiento vertical u horizontal:** El método **pack()** organiza los widgets en un contenedor en una sola dirección: vertical (de arriba a abajo) o horizontal (de izquierda a derecha). Esto se controla mediante el argumento **side**, que puede ser **"top"**, **"bottom"**, **"left"** o **"right"**.
2. **Alineación:** Puedes controlar la alineación de los widgets dentro del contenedor utilizando el argumento **anchor**. Por ejemplo, **"w"** alinea los widgets a la izquierda (oeste), mientras que **"e"** los alinea a la derecha (este).

3. **Relleno:** Puedes especificar cuánto espacio adicional debe agregarse alrededor de un widget utilizando los argumentos **padx** (para el relleno horizontal) y **pady** (para el relleno vertical).
4. **Expansión:** Puedes permitir que un widget se expanda para llenar cualquier espacio adicional disponible en su dirección de empaquetamiento utilizando el argumento **fill**.
5. **Orden de empaquetamiento:** Los widgets se empaquetan en el orden en que se agregan al contenedor. Puedes cambiar este orden utilizando el método **pack()** nuevamente en un widget para moverlo hacia la parte superior o la parte inferior del contenedor.
6. **Widget maestro:** El método **pack()** generalmente se llama en un widget contenedor (por ejemplo, un Frame) para organizar sus widgets secundarios. Los widgets secundarios se agregan al contenedor utilizando otros métodos, como **Label**, **Button**, etc.

Aquí hay un ejemplo simple de cómo se usa **pack()**:

```
import tkinter as tk

root = tk.Tk()

frame = tk.Frame(root)
frame.pack(side="top", padx=10, pady=10, fill="both", expand=True)

label1 = tk.Label(frame, text="Etiqueta 1")
label2 = tk.Label(frame, text="Etiqueta 2")
button = tk.Button(frame, text="Botón")

label1.pack(side="left")
label2.pack(side="left")
button.pack(side="right")

root.mainloop()
```

En este ejemplo, hemos creado un Frame (**frame**) y hemos empaquetado tres widgets (**label1**, **label2**, **button**) dentro de él usando **pack()**. Los argumentos **side**, **padx**, **pady**, **fill** y **expand** se utilizan para controlar la disposición y el espacio en el contenedor.

Eventos en Tkinter

El manejo de eventos en Tkinter, una biblioteca de interfaz gráfica de usuario (GUI) para Python, es fundamental para interactuar con los elementos de la interfaz gráfica y responder a las acciones del usuario, como hacer clic en un botón o ingresar texto en un cuadro de texto. Para manejar eventos en Tkinter, sigue estos pasos generales:

1. **Crear la ventana principal:** Primero, crea la ventana principal de la aplicación utilizando Tk().
2. **Definir funciones de controladores de eventos:** Crea funciones que se ejecutarán cuando ocurra un evento específico. Por ejemplo, si deseas que algo suceda cuando un botón se presione, define una función que contenga ese comportamiento.
3. **Crear widgets:** Crea los widgets (botones, etiquetas, cuadros de texto, etc.) en la ventana principal.
   En este ejemplo, el argumento **command** se utiliza para especificar la función que se llamará cuando se haga clic en el botón.
4. **Empaquetar widgets:** Utiliza el método **pack()** o **grid()** para colocar los widgets en la ventana principal.
5. **Iniciar el bucle principal:** Finalmente, inicia el bucle principal de la aplicación para que la ventana se muestre y los eventos sean manejados.

A continuación, se presentan ejemplos de manejo de eventos para diferentes tipos de widgets en Tkinter:

Manejo de eventos de botón:

```
def boton_clicado():
    label.config(text="¡Botón presionado!")

root = tk.Tk()

label = tk.Label(root, text="")
label.pack()

button = tk.Button(root, text="Presionar", command=boton_clicado)
button.pack()

root.mainloop()
```

Manejo de eventos de entrada de texto (Entry):

```python
def mostrar_texto():
    contenido = entry.get()
    label.config(text=f"Texto ingresado: {contenido}")

root = tk.Tk()

label = tk.Label(root, text="")
label.pack()

entry = tk.Entry(root)
entry.pack()

button = tk.Button(root, text="Mostrar Texto", command=mostrar_texto)
button.pack()

root.mainloop()
```

Con el uso de bind() y trace(), puedes capturar y manejar eventos en un campo de texto Entry de Tkinter. Estos son algunos de los eventos más comunes:

- <Return>: Captura cuando se presiona "Enter".
- <Key>: Captura cualquier pulsación de tecla.
- trace('write'): Detecta cambios en el texto del Entry.
- <FocusIn> y <FocusOut>: Capturan cuando el campo de texto gana o pierde el foco.
- "break": Se usa para evitar que un evento continúe, útil para limitar caracteres.

Ejemplo Return: Evento para controlar cuando se presiona Enter

```
import tkinter as tk
# Crear la ventana principal
root = tk.Tk()
root.title("Evento Enter en Entry")
# Función que se ejecuta al presionar "Enter"
def al_presionar_enter(event):
    texto = entry.get()
    print(f"Texto ingresado: {texto}")
```

```
# Crear un campo de entrada
entry = tk.Entry(root)
entry.pack(pady=10)

# Vincular el evento de presionar "Enter" con la función
entry.bind('<Return>', al_presionar_enter)

# Ejecutar la aplicación
root.mainloop()
```

Ejemplo Evento Obtener y perder foco (FocusIn FocusOut)

```
import tkinter as tk
# Crear la ventana principal
root = tk.Tk()
root.title("Eventos de foco en Entry")
# Función que se ejecuta al obtener el foco
def al_obtener_foco(event):
    print("Entry ha ganado el foco")
# Función que se ejecuta al perder el foco
def al_perder_foco(event):
    print("Entry ha perdido el foco")
# Crear un campo de entrada
entry = tk.Entry(root)
entry.pack(pady=10)
# Vincular los eventos de foco
entry.bind('<FocusIn>', al_obtener_foco)
entry.bind('<FocusOut>', al_perder_foco)

# Ejecutar la aplicación
root.mainloop()
```

Manejo de eventos de listbox:

```
def mostrar_elemento_seleccionado():
    seleccion = listbox.curselection()
    if seleccion:
        indice = seleccion[0]
        elemento = listbox.get(indice)
        label.config(text=f"Elemento seleccionado: {elemento}")
    else:
        label.config(text="Ningún elemento seleccionado")

root = tk.Tk()

label = tk.Label(root, text="")
label.pack()

listbox = tk.Listbox(root)
listbox.pack()

listbox.insert(1, "Elemento 1")
listbox.insert(2, "Elemento 2")
listbox.insert(3, "Elemento 3")

button = tk.Button(root, text="Mostrar Elemento", command=mostrar_elemento_
button.pack()

root.mainloop()
```

Clase TTK

La clase **ttk** en Python se refiere a la biblioteca "themed Tkinter" (ttk), que es una extensión de la biblioteca estándar Tkinter utilizada para crear interfaces gráficas de usuario (GUI) en aplicaciones de escritorio. Tkinter es una biblioteca muy comúnmente utilizada para crear GUI en Python, y ttk mejora y moderniza la apariencia de los widgets de Tkinter.

Las principales ventajas de usar **ttk** son:

1. **Aspecto nativo**: ttk proporciona una apariencia más moderna y nativa en diferentes sistemas operativos, lo que significa que sus aplicaciones tendrán una apariencia más coherente en diferentes plataformas.
2. **Widgets adicionales**: ttk agrega algunos widgets adicionales que no están disponibles en el conjunto estándar de Tkinter, como el Combobox y el Notebook, que son útiles para ciertos tipos de aplicaciones.
3. **Personalización mejorada**: **ttk** ofrece una mayor capacidad de personalización de la apariencia de los widgets, lo que le permite ajustar el aspecto de su GUI de acuerdo con sus necesidades.
4. **Temas**: Puede seleccionar entre diferentes temas o estilos predefinidos para su GUI, lo que facilita la creación de interfaces atractivas sin tener que personalizar cada detalle de la apariencia.
5. **Mejor manejo de fuentes y colores**: ttk proporciona una mejor gestión de fuentes y colores, lo que hace que sea más fácil crear interfaces coherentes y atractivas.

Para utilizar **ttk**, primero debe importarlo en su código, generalmente de la siguiente manera:

```
import tkinter as tk
from tkinter import ttk
```

Controles principales

La biblioteca **ttk** (tema de Tkinter) agrega varios controles y widgets que pueden utilizarse para crear interfaces de usuario modernas y atractivas en aplicaciones de Python. Aquí hay una lista de algunos de los controles más comunes proporcionados por **ttk**:

1. **Botones (Button)**:
    - **ttk.Button**: Un botón con una apariencia moderna y mejorada en comparación con el botón estándar de Tkinter.
2. **Etiquetas (Label)**:
    - **ttk.Label**: Una etiqueta de texto que se utiliza para mostrar información o etiquetar otros widgets.
3. **Entradas de texto (Entry)**:
    - **ttk.Entry**: Un cuadro de entrada de texto que permite al usuario escribir texto o números.
4. **Cuadros de lista (Listbox)**:

- **ttk.Treeview**: Un cuadro de lista que puede mostrar datos en forma de tabla con columnas y filas. Se utiliza comúnmente para mostrar datos tabulares.

5. **Combobox (Combobox)**:
    - **ttk.Combobox**: Un cuadro combinado que combina una entrada de texto con una lista desplegable. Útil cuando se espera que el usuario elija una opción de una lista predefinida.
6. **Cuadernos (Notebook)**:
    - **ttk.Notebook**: Un control que permite crear pestañas en una interfaz de usuario. Útil para organizar contenido en múltiples pestañas.
7. **Barra de progreso (Progressbar)**:
    - **ttk.Progressbar**: Una barra de progreso que muestra el progreso de una tarea en curso.
8. **Radiobotones (Radiobuttons)**:
    - **ttk.Radiobutton**: Botones de opción que permiten al usuario seleccionar una sola opción de un conjunto.
9. **Casillas de verificación (Checkbuttons)**:
    - **ttk.Checkbutton**: Casillas que permiten al usuario seleccionar una o varias opciones de un conjunto.
10. **Barra de desplazamiento (Scrollbar)**:
    - **ttk.Scrollbar**: Una barra de desplazamiento que se utiliza comúnmente junto con otros widgets para permitir la navegación o el desplazamiento de contenido.
11. **Separadores (Separator)**:
    - **ttk.Separator**: Una línea divisoria utilizada para separar visualmente diferentes secciones de una interfaz de usuario.
12. **Árbol (Tree)**:
    - **ttk.Treeview**: Un control que muestra datos en una estructura jerárquica, similar a una estructura de árbol.

Estos son algunos de los controles más utilizados proporcionados por **ttk**. Puede personalizar la apariencia de estos controles y utilizarlos para crear interfaces gráficas atractivas y modernas en sus aplicaciones de Python.

Themes – Temas de TTK

La biblioteca **ttk** de Tkinter proporciona varios temas predefinidos que puedes utilizar para personalizar la apariencia de tus widgets y controles. Estos temas están diseñados para que puedas cambiar rápidamente la apariencia de tu aplicación sin tener que configurar manualmente cada detalle de cada widget.

A partir de la versión 8.5 de Tkinter, los temas disponibles pueden variar según tu versión de Python y Tkinter. Sin embargo, algunos de los temas más comunes que puedes encontrar en versiones modernas de Python incluyen:

1. **clam**: Un tema simple y limpio.
2. **alt**: Un tema alternativo con botones planos y colores suaves.
3. **aqua**: Un tema que sigue las convenciones de apariencia de Aqua en macOS.
4. **default**: El tema predeterminado de la plataforma, que depende del sistema operativo en uso.
5. **classic**: Un tema que imita el aspecto clásico de Tkinter.
6. **vista**: Un tema que sigue el estilo de Windows Vista.
7. **xpnative**: Un tema que sigue el estilo de Windows XP.
8. **winnative**: Un tema que imita la apariencia nativa de Windows.

Puedes seleccionar un tema específico utilizando el método **ttk.Style().theme_use(nombre_del_tema)** antes de crear tus widgets. Por ejemplo, para usar el tema "clam", puedes hacerlo de la siguiente manera:

```
import tkinter as tk
from tkinter import ttk

# Seleccionar el tema "clam"
ttk.Style().theme_use('clam')

# Crear una ventana y widgets ttk
ventana = tk.Tk()
boton = ttk.Button(ventana, text='Botón')
boton.pack()

ventana.mainloop()
```

Ejercicios de practica:

1. Formulario de Registro de Usuario con Validaciones

Crea un formulario de registro de usuario que contenga los siguientes campos:

- Nombre de usuario (Entry)
- Correo electrónico (Entry)
- Contraseña (Entry)
- Confirmar contraseña (Entry)
- Botón de "Registrar"

Requisitos:

1. El nombre de usuario debe contener solo letras y números (sin espacios).
2. El correo electrónico debe tener un formato válido (e.g., nombre@dominio.com).
3. La contraseña debe tener al menos 8 caracteres y contener al menos una letra y un número.
4. La contraseña y la confirmación deben coincidir.
5. Cuando se presione el botón "Registrar", debe aparecer un mensaje indicando si la validación fue exitosa o si hubo errores en alguno de los campos.

---

2. Formulario de Inicio de Sesión con Eventos de Teclado

Crea un formulario de inicio de sesión que tenga los siguientes campos:

- Nombre de usuario (Entry)
- Contraseña (Entry)
- Botón de "Iniciar sesión"

Requisitos:

1. El formulario debe permitir que, al presionar la tecla **"Enter"**, el formulario se envíe automáticamente sin necesidad de presionar el botón.
2. Implementa un evento que, al perder el foco del campo "Nombre de usuario", verifique si este campo está vacío. Si está vacío, debe aparecer un mensaje de advertencia debajo del campo.
3. La contraseña debe ocultarse usando el parámetro show="*" en el campo Entry.

---

3. Formulario de Datos Personales con Validación de Edad

Crea un formulario que recoja los siguientes datos personales:

- Nombre completo (Entry)
- Edad (Entry)
- País (Combobox)
- Género (RadioButtons)
- Botón de "Enviar"

Requisitos:

1. Validar que la edad ingresada sea un número mayor de 18.
2. Implementa un evento que muestre un mensaje de advertencia cuando la edad no sea un número válido.
3. Al hacer clic en "Enviar", muestra un cuadro de diálogo con todos los datos ingresados y valida que los campos no estén vacíos.

---

4. Formulario de Reserva de Hotel con Validación de Fechas

Crea un formulario para una reserva de hotel que contenga:

- Nombre completo (Entry)
- Fecha de check-in (Entry con formato dd/mm/yyyy)
- Fecha de check-out (Entry con formato dd/mm/yyyy)
- Número de personas (Spinbox)
- Botón de "Reservar"

Requisitos:

1. Validar que las fechas de check-in y check-out tengan el formato correcto y que la fecha de check-out sea posterior a la de check-in.
2. Validar que el número de personas sea mayor a 0.
3. Al hacer clic en "Reservar", mostrar una ventana emergente que confirme si los datos son correctos o si hubo algún error en la validación.

---

5. Formulario de Contacto con Eventos Dinámicos

Crea un formulario de contacto que contenga:

- Nombre (Entry)
- Correo electrónico (Entry)

- Mensaje (Text widget)
- Botón de "Enviar"

Requisitos:

1. El campo "Correo electrónico" debe validarse en tiempo real: cuando el usuario escriba, si el correo no tiene el formato correcto, debe aparecer una advertencia en color rojo debajo del campo.
2. El botón "Enviar" solo debe habilitarse si todos los campos han sido llenados correctamente.
3. Al hacer clic en "Enviar", mostrar un cuadro de diálogo que confirme que el mensaje ha sido enviado.

---

6. Formulario de Encuesta con Selección Múltiple y Validación

Crea un formulario de encuesta que permita a los usuarios seleccionar varias opciones:

- Nombre (Entry)
- Género (RadioButtons)
- Opciones de preferencia (Checkbuttons) [Ej: Opción 1, Opción 2, Opción 3]
- Botón de "Enviar"

Requisitos:

1. El campo "Nombre" no debe quedar vacío, y debe validarse que el usuario seleccione al menos una opción en las "Opciones de preferencia".
2. Cuando el formulario se envíe, mostrar los datos ingresados y las opciones seleccionadas en un cuadro de diálogo.
3. Si algún campo no es completado, resaltar los campos faltantes con un borde rojo.